only person who voluntarily declined the prize, true to his existentialist philosophy). In this play, three people — Inèz, Estelle, and Garcin — are condemned to hell for eternity for their sins. They are the only inhabitants of a room in hell where they expect torture. However, there is no physical torture, and after some time, Inèz becomes attracted to Estelle, Estelle becomes attracted to Garcin, and Garcin becomes attracted to Inèz. Thus, everyone is in love but not loved in return for eternity, which is the real torture. Eventually, they come to believe that "Hell is other people" (Sartre's somewhat infamous quote, which, of course, nobody should take literally). Now, let us use a directed graph to denote the relationships between the three characters in the play. The program '*dgraph10.sagews*' shown below generates a graph called *Dgraph_10* (line numbers have been included for clarity in the explanation).

```
1.   DG = DiGraph( )
2.   DG.add_vertices(['Inèz',              'Estelle',
'Garcin'])
3.   DG.add_edges([('Inèz', 'Estelle'),    ('Estelle',
'Garcin'), ('Garcin',     'Inèz')])
4.   DG.plot(vertex_size=1200,             figsize=[8, 8]).
show( )
```

Now, let us go through the code for better understanding. Line 1 initialises an empty directed graph *DG* using SageMath's *DiGraph* class. Line 2 adds three vertices to the graph: '*Inèz*', '*Estelle*', and '*Garcin*'. These vertices represent the three characters in the play '*No Exit*'. Line 3 adds three directed edges to the graph: an edge from '*Inèz*' to '*Estelle*', an edge from '*Estelle*' to '*Garcin*', and an edge from '*Garcin*' to '*Inèz*'. These edges represent the unreciprocated feelings between the three characters. Line 4 plots the directed graph *DG*. The parameter *vertex_size=1200* sets the size of the vertices in the plot, and the parameter *figsize=[8, 8]* sets the size of the plot to 8 x 8 inches. Finally, the *show( )* method displays the plot. Upon execution of the program '*dgraph10.sagews*', we get the graph called *Dgraph_10*. The directed graph *Dgraph_10*, where the directed edges visually depict the direction of each character's attraction towards another, is shown in Figure 3.

Now, let us generate a directed cycle of length 8 using SageMath. We will use this cycle to learn an important difference between a directed graph and an undirected graph. The program '*dcycle11.sagews*' shown below generates a graph called *Dcycle_11*.

```
DG = DiGraph( )
DG.add_vertices([0, 1, 2, 3, 4, 5, 6, 7])
DG.add_edges[(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6),
(6, 7), (7, 0)])
DG.show( )
```

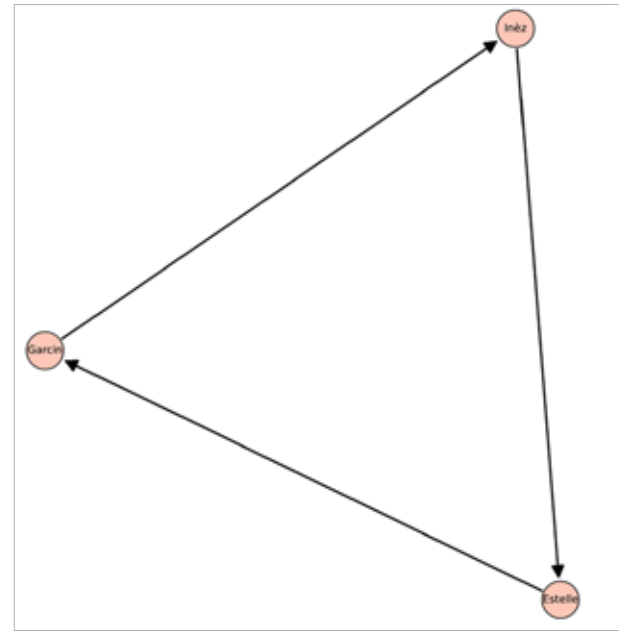

Figure 3: The directed graph *Dgraph_10*

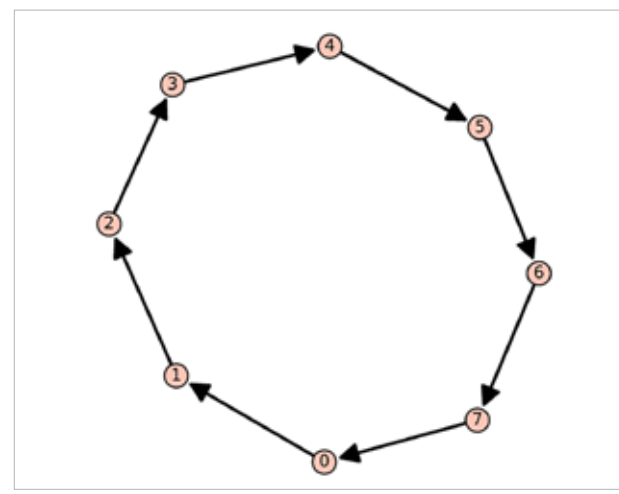

Figure 4: The directed cycle *Dcycle_11* generated by SageMath

The code does not need any explanation, as it is similar to the previous directed graph we have generated. Upon execution of the program '*dcycle11.sagews*', we get the graph called *Dcycle_11*, which is shown in Figure 4.

Now, it is time for us to better understand the main difference between undirected and directed graphs. As seen in the first example, we can represent asymmetric relationships using directed graphs. To illustrate this point further, let us use the *distance( )* method offered by SageMath. This method finds the distance between two vertices. Recall that the distance between a pair of vertices is the number of edges in the shortest path between those two vertices. First, let us test the *distance( )* method on the undirected graph *Disgraph_9*.